Documentación

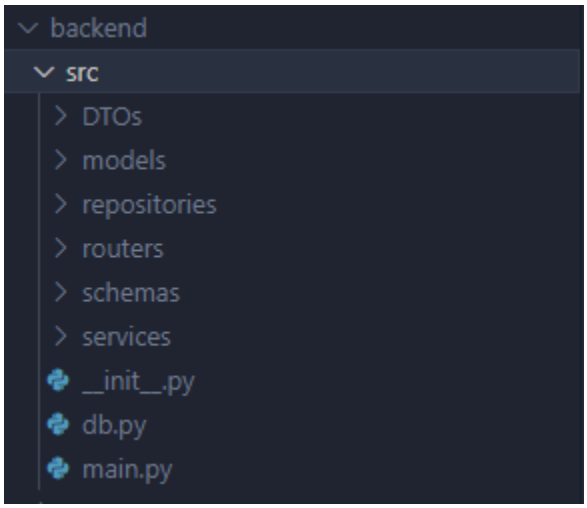
Un ***docstring*** no es más que *un comentario especial ubicado al inicio de una función, cuyo propósito es documentar brevemente su uso y servir como guía para los desarrolladores*. Generalmente se compone de tres partes:

1. **Descripción de la función:** Explica su propósito y contexto de uso.
2. **Descripción de los parámetros:** Detalla los argumentos de entrada, pudiendo incluir pre y post condiciones, así como información extra como los tipos de datos esperados.
3. **Valor de retorno:** Indica qué devuelve la función, con una descripción opcional del resultado y su tipo.

```python
@games_router.post(path="/create", status_code=status.HTTP_201_CREATED)
async def create_game(
    game_info: GameIn, db=Depends(get_db)
) -> GameResponse:
    """
    creates a new game

    Parameters
    ----------
    game_info : GameIn
        Game information
        Player information

    Returns
    -------
    GameResponse
        Game identifier
        Player identifier

    Raises
    ------
    Bad request
        400 -> When there is a game registered with same name
    Internal Server Error
        500 -> Database errors

    Pydantic Errors
    ------
    Error: Unprocessable Content
        422 -> max_players > 6 or min_players < 2 or min_players > max_players or invalid date format
    """
    try:
        result = GameService(db).create_game(game_dto=game_info.to_dto())
```

Organización del proyecto de software

Código Bonito	Nuestro proyecto	Responsabilidad
Capa 0 – Definición de datos	models	Se define como se guardan los datos en la BD
Capa 1 – Acceso a datos	repositories	Operaciones sobre los modelos
Capa 2 – Lógica de aplicación	services	Lógica que ejecutan los endpoints y reglas de negocio
Capa 3 – Interfaz de usuario	routers	Conexión entre el sistema y el mundo exterior
Capa transversal	schemas y DTOs	Define los contratos de entrada/salida (DTOs con Pydantic). Cosas que usan varias capas del sistema.



Testing

Los tests se representan como funciones, y por ello deben tener nombres descriptivos que haga explícito lo que se está probando. Por ejemplo: `test_product_endpoint_raises_error_on_bad_request`, si bien este nombre podría parecer excesivamente largo, en este contexto no hay problema, lo importante es que sean precisos.

test_create_game.py

C: > Users > Usuario > Downloads > test_create_game.py

```python
import pytest
from conftest import client

@pytest.fixture(scope="module")
def test_client():
    yield client

def test_create_game_success(test_client): ...

def test_create_game_invalid_date(): ...

def test_create_game_empty_fields(): ...

def test_create_game_bad_field_type(): ...

def test_create_game_minPlayers_greater_than_maxPlayers(): ...

def test_create_game_minPlayers_maxPlayers_out_of_valid_range(): ...
```

Tests de integración

El segundo nivel de la pirámide corresponde a los tests de integración. A diferencia del nivel anterior, donde se validaban simplemente piezas de código aisladas, los tests de integración se enfocan en verificar cómo se relacionan e interactúan diferentes componentes del sistema. Su objetivo es asegurarse de que las partes del sistema colaboran correctamente respetando el flujo de datos.

Una herramienta comúnmente utilizada en este tipos de pruebas son los *fixtures*, proporcionados en Python por librerías como `pytest`. Los *fixtures* permiten definir un entorno de pruebas que se prepara antes (y opcionalmente después) de ejecutar cada prueba. Esto los hace ideales para inicializar datos, establecer conexiones o limpiar recursos, asegurando que cada prueba se ejecute en un contexto controlado y repetible.

```python
def test_create_game_success(test_client):

    response = test_client.post(
        "games/create",
        json={
            "game_name": "prueba2",
            "min_players": 2,
            "max_players": 6,
            "player_name": "Nombre1",
            "player_birth_date": "2025-09-21"
        },
    )
    assert response.status_code == 201
    data = response.json()
    assert "game_id" in data
    assert "player_id" in data
```

```python
def test_create_game_invalid_date():
    response = client.post(
        "/games/create",
        json={
            "game_name": "prueba1",
            "min_players": 2,
            "max_players": 6,
            "player_name": "Nombre1",
            "player_birth_date": "ee223dd"
        },
    )
    assert response.status_code == 422

    error_detail = response.json()["detail"]
    assert isinstance(error_detail, list)
    assert "Input should be a valid date or datetime" in error_detail[0]["msg"]
```

En el texto decía que los tests de integración son los que prueban cómo las piezas del sistema se relacionan entre sí, a diferencia de los unitarios, que chequean funciones solitas. La idea es asegurarse de que el flujo de datos viaja bien de una capa a otra.

- test_create_game_success prueba el caso exitoso, enviando un request válido y se testea router → service → validaciones → repositorio/modelos
- invalid_date, empty_fields, bad_field_type, etc. Prueba las validaciones de Pydantic.
- No son tests aislados
- Se usó un fixture test_client para levantar un entorno controlado